PRODUCT REQUIREMENTS DOCUMENT

# Explainable House Price Prediction Using R and mlr3

*End-to-end R data science, machine learning, explainability and Shiny application*

Master Project Specification
Version 1.0
5 October 2026

**Final one-line project statement**

Develop an explainable house-price prediction system in R using mlr3, SHAP/LIME, and Shiny, with comprehensive preprocessing, modeling, and evaluation.

Primary focus: R data science + preprocessing + machine learning
Secondary focus: explainability + interactive deployment

# Document Contents

1. Executive Summary

2. Project Definition and Problem Statement

3. Goals, Objectives and Success Criteria

4. Dataset and Data Provenance

5. Functional Requirements

6. Non-Functional Requirements

7. Solution Architecture and End-to-End Workflow

8. Data Understanding and Preprocessing Plan

9. Feature Engineering and Selection

10. Machine Learning Strategy

11. Model Evaluation and Experimental Design

12. Explainable AI Strategy: SHAP and LIME

13. Shiny Application Requirements

14. Technology Stack and Package Plan

15. Research Foundation and Paper Review

16. Research-to-Implementation Traceability

17. Implementation Limitations and Risk Controls

18. Future Scope

19. Phased Execution Plan

20. Repository and Deliverables

21. Acceptance Criteria

22. References

# 1. Executive Summary

**Purpose.** Build a reproducible, explainable regression system in R that predicts residential sale prices from the Ames Housing dataset while using the project to learn and demonstrate the full practical workflow of R-based machine learning.

- Core learning outcome: comprehensive data inspection, cleaning, missing-value handling, categorical/numerical preprocessing, transformation, feature engineering, feature selection, model training, resampling and tuning.
- Primary ML framework: mlr3, extended by mlr3pipelines for train/predict-aware preprocessing graphs and mlr3tuning for optimization and nested resampling.
- Explainability layer: global and local explanations using SHAP and LIME, with explicit comparison of what each method explains and its limitations.
- Application layer: an interactive Shiny application that accepts house characteristics, returns an estimated sale price and presents the major factors influencing that prediction.
- Academic principle: do not fabricate data problems or results; document which data issues are present in the adapted dataset and distinguish source-documented issues from project-created feature engineering decisions.

# 2. Project Definition and Problem Statement

## 2.1 Problem

Residential property prices depend on structural, locational, quality, condition and transaction attributes. The project addresses the supervised regression problem of estimating a property' s sale price from these attributes and demonstrating a robust, explainable ML workflow rather than building a black-box predictor only.

## 2.2 Proposed Solution

Create a single reproducible pipeline in R that performs data-quality checks and preprocessing inside the ML workflow where appropriate, trains and benchmarks multiple regression learners, tunes the strongest candidates, evaluates generalization with resampling, and exposes the final fitted workflow through Shiny with local explanations.

## 2.3 Intended Users

- Primary: academic evaluator, instructor or reviewer assessing R/ML methodology.
- Secondary: ML/data-science recruiter reviewing the portfolio project.
- Application demonstrator: a user who wants to estimate a plausible sale price for a house profile and understand influential attributes.

## 2.4 Scope Boundary

| In scope | Phase 1 out of scope |
|---|---|
| EDA, data-quality audit, missing-data handling, encoding, transformations, scaling where justified, outlier analysis, feature engineering, feature selection, model benchmarking, CV, tuning, SHAP, LIME, Shiny. | Live market pricing, external real-estate APIs, authentication/authorization, multi-user production operations, automated cloud scaling, full MLOps/CI-CD, real-time retraining. |
| Reproducible R code, documented experiments, final model selection and an interactive demonstration. | A claim that predicted values are professional appraisals or investment advice. |

# 3. Goals, Objectives and Success Criteria

## 3.1 Primary Objectives

**1.** Learn and apply a complete R data-science workflow on a non-trivial tabular dataset.

**2.** Implement preprocessing in a leakage-safe way using mlr3/ mlr3pipelines where model-dependent transformations are required.

**3.** Train and compare diverse regression models rather than selecting a model by intuition alone.

**4.** Use resampling and hyperparameter tuning to estimate and improve generalization performance.

**5.** Produce both global and individual explanations for model predictions.

**6.** Package the final workflow into an interactive Shiny application.

## 3.2 Secondary Objectives

- Develop reusable R modules and a professional GitHub repository.
- Create a clear experiment log so every model and preprocessing choice is traceable.
- Document limitations, assumptions, ethical considerations and future improvements.

## 3.3 Success Criteria

| Area | Minimum acceptance condition |
|---|---|
| Data | Dataset loads reproducibly; structure, missingness, types, duplicates, constants, redundant features and suspicious encodings are audited. |
| Preprocessing | All model-dependent preprocessing required for evaluation is encapsulated in the training workflow to reduce leakage risk. |
| Models | At least four distinct regression approaches are benchmarked, including a simple baseline and at least one tree/ensemble method. |
| Validation | Cross-validation is performed and a separate final test/holdout evaluation is retained when the experimental design permits. |
| Tuning | At least one major model is tuned with a documented search space and reproducible budget/terminator. |
| XAI | Both SHAP and LIME produce interpretable local explanations for selected instances; a global importance view is also provided. |

| Area | Minimum acceptance condition |
|---|---|
| Shiny | A user can enter/select a house profile and receive a prediction plus explanatory output. |
| Reproducibility | All installation, data acquisition and analysis steps are documented; results can be regenerated without hidden manual transformations. |

# 4. Dataset and Data Provenance

## 4.1 Selected Dataset

**Dataset:** `mlr3data::ames_housing`, the adapted Ames Housing dataset used in the official Applied Machine Learning Using mlr3 in R preprocessing material.

| Property | Project specification |
|---|---|
| Observations | 2,930 residential properties |
| Predictors | 80 predictor features in the mlr3data reference |
| Target | `Sale_Price` (USD) |
| Task type | Supervised regression |
| Geography | Ames, Iowa, USA |
| Period | Sales from 2006 to 2010 |
| Feature mix | Numerical/discrete and categorical variables covering property size, structure, quality, condition, amenities and transaction information |
| Key reason for selection | The mlr3 book explicitly uses an adapted version and introduces additional artificial problems so many preprocessing concepts can be demonstrated on one dataset. |

**Important provenance note**

The project must use the mlr3data adaptation, not an arbitrary cleaned Kaggle copy. The adaptation is intentionally

educational: some problematic features were introduced artificially. Those changes are part of the documented teaching setup and should be described as such in the final report.

## 4.2 Main Documented Data-Quality Cases

| Issue | Example documented in the mlr3 material | Planned handling |
|---|---|---|
| Constant feature | `Misc_Feature_2` has one level | Remove using explicit audit or PipeOpRemoveConstants |
| Duplicate feature | `Condition_2` and `Condition_3` are identical | Remove one and document redundancy |
| Same information on different scales | `Lot_Area` and `Lot_Area_m2` are perfectly correlated | Retain one representation |
| Missing data | Several variables contain NAs, including `Alley` and basement-related variables | Compare appropriate imputation strategies; preserve missingness semantics where useful |
| Potential semantic problems | The material recommends checking IDs, incorrectly encoded NAs and semantic errors such as negative areas | Audit raw values before modeling |
| Categorical encoding | High-cardinality factors include variables such as `Exterior_1st`, `Exterior_2nd`, `MS_SubClass`, and `Neighborhood` | Compare/tune encoding strategies |

## 4.3 Acquisition and Loading

1. Install `mlr3`, `mlr3data`, and the required extensions.
2. Load the task/data directly from the package for the canonical project dataset.
3. Record package versions with `sessionInfo()` and preferably `renv`.
4. Export a project copy only when needed for inspection or external processing; do not modify the canonical source silently.

```
install.packages(c("mlr3", "mlr3data"))
library(mlr3)
library(mlr3data)
task<-tsk("ames_housing")
data<-task$data()
```

# 5. Functional Requirements

| ID | Requirement | Description |
|---|---|---|
| FR-01 | Data ingestion | Load the canonical Ames task/data reproducibly. |
| FR-02 | Data profiling | Report shape, types, missingness, cardinality, distributions, duplicates and basic descriptive statistics. |
| FR-03 | Data-quality audit | Identify constants, duplicate/redundant variables, IDs, suspicious encodings and semantic/range issues. |
| FR-04 | Preprocessing | Implement required cleaning, imputation, encoding, transformation and scaling as documented, leakage-safely. |
| FR-05 | Feature engineering | Create only defensible derived variables supported by domain logic and evaluate whether they improve validation performance. |
| FR-06 | Model benchmarking | Benchmark baseline, linear/regularized, tree and ensemble learners using common resampling and measures. |
| FR-07 | Hyperparameter tuning | Tune selected models with mlr3tuning and retain the tuning archive/results. |
| FR-08 | Model selection | Select a final candidate based on predefined performance criteria and interpretability/computational trade-offs. |
| FR-09 | Explainability | Generate SHAP and LIME explanations for representative individual cases and aggregate feature influence. |
| FR-10 | Shiny prediction | Allow interactive house input and return a predicted sale price. |
| FR-11 | Shiny explanation | Display the factors driving the selected prediction and clearly label explanations as model-based, not causal. |
| FR-12 | Reproducibility | Provide documented setup, versioning, deterministic seeds where applicable and one-command/one-script execution where feasible. |

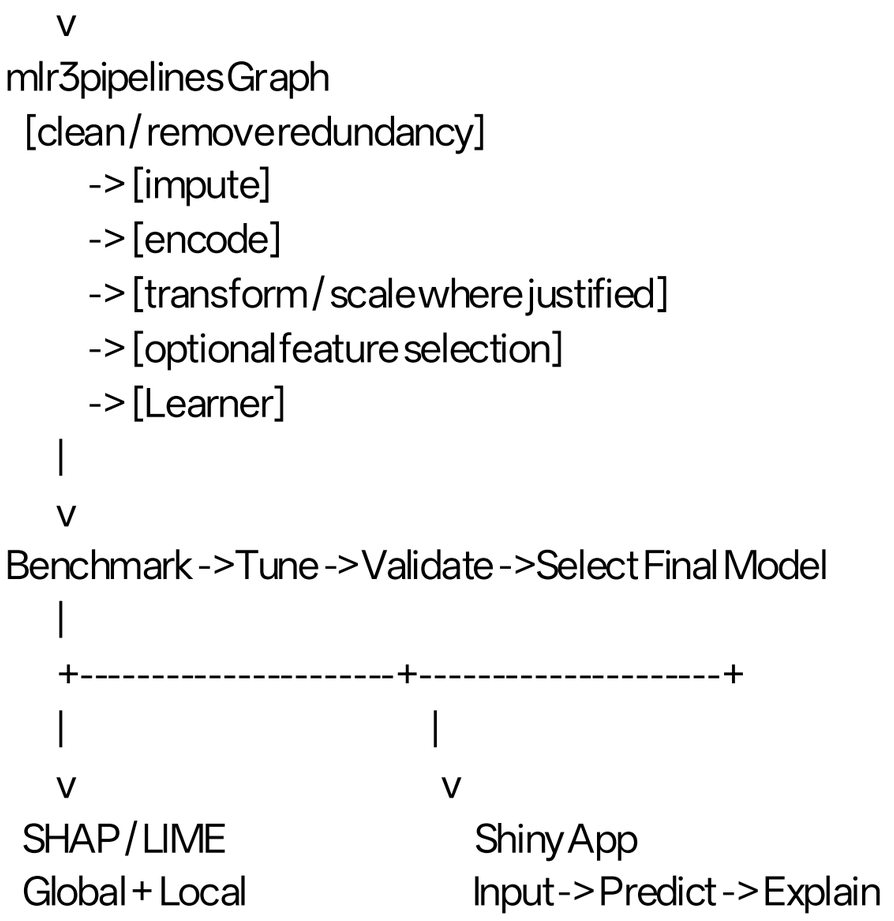
# 6. Non-Functional Requirements

| ID | Requirement | Acceptance standard |
|---|---|---|

| ID | Requirement | Acceptance standard |
|---|---|---|
| NFR-01 | Reproducibility | Same dataset, package versions and seed should reproduce analysis decisions within expected stochastic variation. |
| NFR-02 | Leakage resistance | Preprocessing parameters are learned from training data within the resampling workflow. |
| NFR-03 | Usability | Notebook/report explains the purpose and interpretation of each major analysis step. |
| NFR-04 | Maintainability | Reusable logic is separated into scripts/functions rather than hidden inside large monolithic cells. |
| NFR-05 | Traceability | Each experiment has a name, configuration, evaluation metric and result record. |
| NFR-06 | Interpretability | Explainability output must distinguish association/attribution from causation. |
| NFR-07 | Performance | Shiny prediction should feel interactive for a single input; expensive explanation calculations may be triggered on demand. |
| NFR-08 | Academic integrity | No fabricated model metrics, artificially invented data defects, or unsupported claims about causality or production accuracy. |

# 7. Solution Architecture and End-to-End Workflow

Ames Housing
|
v
Data Audit -> EDA -> Feature/Target Definition
|
v
mlr3 Task + Resampling Strategy
|




## 7.1 Architectural Principle

A preprocessing step that learns parameters from the data should live inside the resampled pipeline whenever possible. This prevents information from the validation/test partition leaking into training-time transformations and is a central reason to use mlr3pipelines instead of manually preprocessing the full dataset first.

## 7.2 Proposed Pipeline Families

| Pipeline | Purpose |
|---|---|
| Baseline | Featureless/constant predictor to establish a lower performance reference. |
| Linear | Minimal preprocessing + linear regression for interpretability and a transparent reference point. |
| Regularized | Encoding + scaling where needed + Elastic Net/Lasso-style regularization to test feature shrinkage. |
| Tree | Decision tree with minimal scaling because tree splits do not require standardization. |
| Ensemble | Random forest / extremely randomized tree approach for nonlinear interactions and robust predictive performance. |
| Boosting | Gradient-boosted trees such as XGBoost when included and properly encoded; expected to be a strong performance candidate. |

# 8. Data Understanding and Preprocessing Plan

## 8.1 Data Audit Checklist

- Dataset dimensions, target distribution and row/column identifiers.
- Data types and factor levels; identify numeric values represented as categorical and vice versa.
- Missingness by variable and, where useful, co-missingness patterns.
- Constant and near-constant features.
- Exact duplicated variables and highly correlated/redundant variables.
- Potential ID columns and variables that could encode the row identity rather than generalizable signal.
- Range/semantic checks (e.g., negative areas, implausible year relationships).
- Skewness, heavy tails and influential observations, especially for `Sale_Price`.

## 8.2 Preprocessing Techniques to Demonstrate

| Technique | Examples / purpose |
|---|---|
| Missing-value handling | Mean/median/mode where justified; histogram/distribution sampling; out-of-range imputation; explicit missing category for categorical semantics; missingness indicators. |
| Categorical encoding | One-hot/treatment encoding; collapse rare levels; impact encoding as an advanced experiment for suitable models. |
| Transformation | Investigate log transformation of skewed numeric predictors and potentially the target; compare before/after distributions. |
| Scaling | Center/scale numeric features for models that benefit from comparable scales; avoid unnecessary scaling for trees. |
| Outlier handling | Detect with boxplots/IQR or robust statistics; compare retain, transform or winsorize only when justified and without leaking test information. |
| Feature removal | Remove constants, exact duplicates and redundant duplicate-scale representations. |
| Target transformation | Evaluate log-sale-price modeling where statistically justified; invert transformation at prediction time and report metrics on the original price scale. |
| Feature selection | Use correlation/filtering and optionally model-based feature filtering; verify any selected subset inside resampling when selection is data-dependent. |

### 8.3 Data Leakage Controls

- Do not compute global means/medians/encoders using the full dataset before resampling.
- Do not select features using the full outcome-labeled dataset and then report cross-validation as if the selection were independent.
- Keep test/holdout data untouched until the final evaluation stage.
- For transformations derived from training data, fit them only inside the pipeline training phase.
- Document any feature that could leak transaction-order or post-sale information.

## 9. Feature Engineering and Selection

### 9.1 Candidate Engineered Features

- Total finished living/usable area aggregates where component areas are logically additive.
- Total bathroom count using weighted half-bath contribution.
- Total porch/deck area where semantically appropriate.
- Age-at-sale features using construction/remodel year and sale year, with explicit treatment of missing year fields.
- Garage/building age or recency features where the underlying dates are valid.
- Quality × size interactions for a limited set of high-value variables if supported by EDA.
- Log transforms of heavily right-skewed size/value features, where validated.

**Feature-engineering rule**

Every engineered feature must have a short justification, a defined formula, a documented data-dependency path, and an empirical validation result. Feature creation is not automatically an improvement.

### 9.2 Feature Selection Experiments

- Remove constant/duplicate/redundant variables.
- Correlation-based screening with careful interpretation of categorical variables.
- Model-based importance as a diagnostic, not as the sole criterion for causal relevance.
- Permutation-based importance on a held-out/resampled evaluation set for more reliable generalization interpretation.
- Compare full-feature and selected-feature models on the same resampling scheme.

## 10. Machine Learning Strategy

| Model family | Role | Why included | Preprocessing sensitivity |
|---|---|---|---|
| Featureless regression | Baseline | Defines the minimum reference performance. | Very low |
| Linear regression | Transparent baseline | Connects R statistics/regression concepts to the ML workflow. | Moderate for factors/skew/scaling depending on specification |

| Model family | Role | Why included | Preprocessing sensitivity |
|---|---|---|---|
| Elastic Net / glmnet | Regularized model | Demonstrates shrinkage, multicollinearity handling and the value of scaling/encoding. | High |
| Decision tree | Interpretable nonlinear model | Captures thresholds/interactions and is easy to visualize. | Low |
| Random forest / ranger | Strong ensemble | Handles nonlinear structure and interactions with limited manual scaling. | Low to moderate |
| XGBoost | Advanced benchmark | Strong boosted-tree candidate and useful for comparing preprocessing requirements. | Moderate to high because categorical encoding is required |

### 10.1 Model Selection Principle

The final model is not simply the one with the lowest single metric. The decision should consider cross-validated error, stability across folds, computational cost, sensitivity to preprocessing, interpretability, explanation quality and the intended Shiny user experience.

## 11. Model Evaluation and Experimental Design

### 11.1 Primary Metrics

| Metric | Purpose | Decision use |
|---|---|---|
| MAE | Average absolute prediction error in USD. | Primary human-interpretable error measure. |
| RMSE | Penalizes larger errors more strongly. | Secondary measure for expensive large mistakes. |
| R-squared | Explains proportion of outcome variance represented by the model. | Contextual model-fit measure; not a substitute for error in USD. |

| Metric | Purpose | Decision use |
|---|---|---|
| MAPE | Relative percentage error. | Optional supplemental metric; interpret carefully near zero targets (not a practical issue for positive house sale prices but still document assumptions). |

## 11.2 Validation Design

- Use a reproducible train/test or train/holdout split for the final untouched evaluation when feasible.
- Use k-fold cross-validation within the training data for model benchmarking.
- Use nested resampling for the strongest tuning experiments when computationally practical.
- Use identical resampling splits when comparing candidate models so differences are attributable to the models/pipelines rather than different folds.
- Report mean performance and variability across folds rather than only the best fold.

## 11.3 Experiment Matrix

| Experiment | Question |
|---|---|
| E1: Baseline vs linear | How much predictive signal exists beyond a constant predictor? |
| E2: Linear + richer preprocessing | Do encoding/transformation choices materially improve a linear model? |
| E3: Tree vs linear | How much nonlinear structure is present? |
| E4: Random forest | Does an ensemble substantially reduce validation error? |
| E5: Boosting | Does boosting outperform the strongest tree baseline? |
| E6: Full vs selected features | Does feature selection reduce complexity without harming generalization? |
| E7: Target transformation | Does log-target modeling improve the error profile after back-transformation? |
| E8: Tuned vs default | How much gain comes from hyperparameter optimization? |
| E9: XAI consistency | Do SHAP and LIME agree on the dominant contributors for selected cases? |

# 12. Explainable AI Strategy: SHAP and LIME

## 12.1 Why Explainability Is Required

A high-performing model is not sufficient for this project because one objective is to understand model behavior. Explanations will help show which features drive predictions, compare explanation methods, identify surprising behavior, and make the final Shiny app educational rather than merely predictive.

## 12.2 SHAP

- Generate per-feature contribution values for an individual house prediction.
- Aggregate absolute SHAP values across a sample to obtain a global importance ranking.
- Present local contributions as positive/negative influence relative to the model baseline.
- Use a wrapper compatible with the final mlr3 learner and ensure preprocessing applied during explanation matches the trained model workflow.

## 12.3 LIME

- Generate a local surrogate explanation around a selected house instance.
- Display the small set of features with the strongest local effects in the neighborhood considered by LIME.
- Compare selected LIME results with SHAP for the same instance to discuss agreement and differences.

## 12.4 Explainability Constraints

- Treat SHAP/LIME as explanations of model behavior, not causal explanations of real-world house prices.
- Document background/reference data choices for SHAP and perturbation/sampling choices for LIME.
- Avoid showing unexplained precision such as "this feature caused the price by exactly X dollars" unless the method and interpretation support that claim.
- Cache expensive explanation artifacts when possible; make on-demand explanation optional in Shiny.

# 13. Shiny Application Requirements

## 13.1 Application Goal

Create a browser-based R application that turns the trained regression workflow into an interactive demonstration: users provide a house profile, the application returns a predicted price, and the application explains which factors influenced that prediction.

## 13.2 Proposed UI

| View | Core elements |
|---|---|
| Home / Overview | Project purpose, dataset scope, model disclaimer and workflow summary. |
| Prediction | House attributes grouped into logical sections such as property, quality, size, basement, garage and sale context. |

| View | Core elements |
|---|---|
| Result | Predicted sale price, optional prediction interval/uncertainty if later implemented, and a concise explanation summary. |
| Explainability | SHAP local bar/force-style visualization and LIME local contribution table/plot. |
| Model / Method | Selected model name, validation summary and high-level methodology. |
| Data / Insights | Optional EDA charts and dataset information for project demonstration. |

### 13.3 Input Strategy

Because the canonical dataset has many predictors, the first UI should expose the most decision-relevant variables in a practical form while offering an Advanced mode for full-feature access. Any defaulted fields must be clearly documented; silent hidden assumptions are not acceptable.

### 13.4 Shiny Technical Design

- Use `app.R` as the application entry point with explicit UI and server components.
- Load the final model/pipeline and explanation configuration once per application session where practical.
- Validate user inputs before scoring; show useful validation messages.
- Separate reactive data assembly, prediction, and explanation calculations to keep the server logic maintainable.
- Do not refit the model inside the prediction handler.
- Make explanation generation conditional so ordinary predictions remain responsive.

## 14. Technology Stack and Package Plan

| Layer | Recommended technology | Purpose |
|---|---|---|
| Language | R | Primary language and statistical computing environment. |
| Core ML framework | mlr3 | Tasks, learners, resampling, measures, benchmarking. |
| Pipelines | mlr3pipelines | DAG-based preprocessing/model graphs and leakage-aware train/predict transformations. |

| Layer | Recommended technology | Purpose |
|---|---|---|
| Tuning | mlr3tuning | Hyperparameter optimization and nested resampling. |
| Data package | mlr3data | Canonical adapted Ames Housing dataset. |
| Feature filtering | mlr3filters | Optional/model-independent feature filtering experiments. |
| Data manipulation | data.table and/or dplyr | Inspection and transparent data transformations outside model-trained pipeline state. |
| Visualization | ggplot2 | EDA, diagnostics and result visualization. |
| Explainability | SHAP-capable R tooling plus `lime` or another documented R implementation | Global/local explanations; exact package choice should be frozen in `renv` once implementation begins. |
| Web application | shiny + bslib | Interactive prediction and explanation interface. |
| Reproducibility | renv | Package/version lock and environment restoration. |
| Version control | Git + GitHub | Source control, issues and project presentation. |

**Framework discipline**

Use mlr3 as the central ML framework. Avoid mixing several end-to-end ML frameworks such as caret/tidymodels for the same modeling pipeline unless a later comparison is explicitly added as a separate research experiment.

## 15. Research Foundation and Paper Review

The project is grounded primarily in the development of the mlr family and its pipeline extension, with the adapted Ames dataset documented in the official mlr3 book. The papers below are used as methodological foundations, not as claims that the project reproduces the research contributions of those papers.

## 15.1 Paper A — mlr: Machine Learning in R (2016)

| Item | Details |
|---|---|
| Title | mlr: Machine Learning in R |
| Authors | Bernd Bischl, Michel Lang, Lars Kotthoff, Julia Schiffner, Jakob Richter, Erich Studerus, Giuseppe Casalicchio, Zachary M. Jones |
| Venue | Journal of Machine Learning Research |
| Citation | JMLR 17(170):1– 5, 2016 |
| Core contribution | A generic, object-oriented and extensible framework for classification, regression, survival analysis and clustering in R. |
| Relevant capabilities | Unified learner interface, resampling, hyperparameter tuning, feature selection, ensemble methods and parallel high-performance computing. |
| Project relevance | Provides the historical design foundation for the mlr family and motivates a unified experiment-oriented ML workflow. |

## 15.2 Paper B — mlr3: A Modern Object-Oriented Machine Learning Framework in R (2019)

| Item | Details |
|---|---|
| Title | mlr3: A modern object-oriented machine learning framework in R |
| Authors | Michel Lang, Martin Binder, Jakob Richter, Patrick Schratz, Florian Pfisterer, Stefan Coors, Quay Au, Giuseppe Casalicchio, Lars Kotthoff, Bernd Bischl |
| Venue | Journal of Open Source Software |
| Citation | JOSS 4(44), 1903, 2019 |
| DOI | 10.21105/joss.01903 |

| Item | Details |
|---|---|
| Core contribution | A complete reimplementation of mlr with a modular, extensible architecture and unified support for training/evaluation, resampling, preprocessing, tuning, nested resampling and visualization. |
| Design principles | Modular/light dependencies, efficient data.table usage, R6 object orientation, defensive programming and type safety. |
| Project relevance | This is the primary framework reference for the project architecture and experimental workflow. |

## 15.3 Paper C — mlr3pipelines: Flexible Machine Learning Pipelines in R (2021)

| Item | Details |
|---|---|
| Title | mlr3pipelines - Flexible Machine Learning Pipelines in R |
| Authors | Martin Binder, Florian Pfisterer, Michel Lang, Lennart Schneider, Lars Kotthoff, Bernd Bischl |
| Venue | Journal of Machine Learning Research |
| Citation | JMLR 22(184):1– 7, 2021 |
| Core contribution | A dataflow framework for representing linear and complex nonlinear ML workflows as directed acyclic graphs. |
| Relevant capabilities | Composable PipeOps, branching, ensembles/stacking patterns, preprocessing and model fitting, joint hyperparameter exposure and leakage-safe train/predict state. |
| Project relevance | Directly supports the project’ spreprocessing + learner graph and the requirement that transformations be learned only from training data. |

## 15.4 Dataset/Teaching Reference — Applied Machine Learning Using mlr3 in R

| Item | Details |
|---|---|

| Item | Details |
|---|---|
| Editors | Bernd Bischl, Raphael Sonabend, Lars Kotthoff, Michel Lang |
| Publisher / year | CRC Press, 2024 |
| Relevant chapter | Chapter 9: Preprocessing |
| Ames contribution | Uses an adapted Ames Housing dataset and intentionally introduces additional artificial problems so many preprocessing concepts can be demonstrated on one dataset. |
| Project relevance | Provides the closest pedagogical blueprint for our selected dataset and the preprocessing objectives. |

### 15.5 What the Papers Do — and Do Not — Claim

The papers are framework/software papers. They establish software architecture and capabilities; they do not claim that using mlr3 automatically produces the best model for a particular dataset, nor do they validate SHAP/LIME explanations for this project. Our empirical results must therefore come from our own controlled experiments.

## 16. Research-to-Implementation Traceability

| Research idea | Project implementation | Evidence of completion |
|---|---|---|
| Unified ML interface | Represent the housing problem as an mlr3 regression Task and use Learner objects. | Task/learner setup script + benchmark output. |
| Resampling | Use k-fold CV and, where appropriate, nested resampling. | Stored ResampleResult / BenchmarkResult and evaluation table. |
| Preprocessing pipelines | Compose imputation, encoding, transformation and selection PipeOps. | Pipeline graph and train/predict demonstrations. |
| No leakage from preprocessing state | Fit learned preprocessing within resampling/pipeline training stage. | Methodology note + code inspection + comparison with naive preprocessing as a cautionary experiment if useful. |
| Hyperparameter tuning | Use mlr3tuning with a documented search space. | Tuning archive + tuned configuration + outer resampling estimate. |

| Research idea | Project implementation | Evidence of completion |
|---|---|---|
| Feature filtering | Use selected filters or explicit data-quality rules. | Before/after feature set and performance comparison. |
| Experiment benchmarking | Compare all candidates using common resampling folds and metrics. | Benchmark table and visual comparison. |
| Complex workflow extensibility | Optionally test branch/alternative preprocessing paths. | Pipeline graph demonstrating alternative preprocessing/model path. |

# 17. Implementation Limitations and Risk Controls

## 17.1 Research / Framework Limitations

The mlr3 paper does not present a conventional "limitations" section. Its discussion instead explains why the older mlr architecture became difficult to maintain and how mlr3 addresses those lessons. Those historical limitations should not be misreported as limitations of our implementation.

| Limitation / consideration | Impact on this project | Control / response |
|---|---|---|
| Modular ecosystem | Useful capabilities are distributed across multiple extension packages, increasing dependency management. | Use `renv`; record package versions; keep the core dependency list minimal. |
| Pipeline complexity | Complex DAGs are powerful but harder for beginners to debug. | Build the pipeline incrementally and inspect each PipeOp before composing the full graph. |
| Learner compatibility | Different learners support different data types and require different preprocessing. | Document learner properties and create model-specific pipeline branches when justified. |
| Computational cost of nested tuning | Nested resampling can become expensive with many models/features. | Use a staged strategy: broad benchmark first, then deeply tune only a small set of finalists. |
| SHAP/LIME implementation overhead | Explanations add computation and depend on careful model wrappers/reference data. | Explain only selected instances on demand; cache reusable objects; validate output against known cases. |

## 17.2 Dataset and Modeling Limitations

| Limitation | Why it matters |
|---|---|
| Historical/geographic specificity | The data describes Ames, Iowa, sales from 2006– 2010; current markets or other geographies may behave differently. |
| Educational adaptation | The mlr3 version includes intentionally introduced artificial problems. These are excellent for preprocessing education but are not evidence of naturally occurring data-quality rates. |
| Regression target limitations | Sale price is an observed transaction outcome, not a true latent market value or professional appraisal. |
| Potential temporal dependence | Random CV may not fully represent future-market generalization if year effects are used. A temporal robustness experiment is recommended as an extension. |
| Feature semantics | Some property variables are categorical ratings with domain-specific meaning; naive numerical treatment can be misleading. |
| Model explanation limits | SHAP/LIME describe model attribution/local behavior; they do not establish causality. |
| Shiny input burden | Exposing all predictors creates a cumbersome user experience. The Basic/Advanced input strategy is therefore required. |

## 17.3 Major Project Risks

| Risk | Likelihood | Mitigation |
|---|---|---|
| Data leakage through preprocessing | High | Put learned transforms inside mlr3pipelines and explicitly test methodology. |
| Over-tuning / compute explosion | Medium | Use staged model screening, narrow tuning spaces and sensible evaluation budgets. |
| Poor explanation consistency | Medium | Compare SHAP/LIME on representative cases and document differences rather than forcing agreement. |

| Risk | Likelihood | Mitigation |
|---|---|---|
| Shiny becomes a distraction from ML | Medium | Freeze the predictive pipeline before building the UI; treat Shiny as the final layer. |
| Scope creep | High | Keep API/cloud/MLOps as future scope until core research objectives are completed. |
| Unsupported claims | Medium | Label dataset limitations and explainability limitations explicitly in the report. |

## 18. Future Scope

Future work should extend the project without weakening its core methodology. The mlr3 paper itself described planned ecosystem extensions including Bayesian optimization, Hyperband, probabilistic regression, survival analysis and spatial/temporal data support; later mlr3 ecosystem releases provide several of these capabilities through extension packages. The project can use this ecosystem direction as a roadmap.

| Future enhancement | Value |
|---|---|
| Bayesian optimization / Hyperband | Reduce tuning cost and explore more advanced hyperparameter optimization. |
| Nested, multi-objective tuning | Optimize predictive error together with complexity or computational cost. |
| Stacking / ensembles | Combine complementary learners for potentially improved predictive performance. |
| Probabilistic prediction / uncertainty | Provide uncertainty or prediction intervals rather than only point estimates. |
| Temporal validation | Train on earlier sales and validate on later sales to study temporal generalization. |
| Spatial features | Use location-derived variables carefully to capture neighborhood/spatial effects. |
| Counterfactual / what-if interface | Allow users to modify a property characteristic and inspect how the predicted price changes, with careful non-causal wording. |

| Future enhancement | Value |
|---|---|
| Advanced Shiny deployment | Add authentication, API separation, containerization, cloud hosting, logging and monitoring for a production-style system. |
| Automated retraining / MLOps | Schedule data refresh, model validation, drift checks and controlled model promotion. |
| Model governance | Add model cards, reproducibility metadata, explanation audit trails and explicit limitations. |

## 19. Phased Execution Plan

| Phase / week | Main work | Deliverables |
|---|---|---|
| Week 1 - Foundations | Install R ecosystem; study papers; acquire/load dataset; define task, target and research questions; initial EDA and quality audit. | Project brief, literature notes, data dictionary, initial EDA. |
| Week 2 - Preprocessing | Missingness, data types, constants, duplicates/redundancy, categorical encoding, transformations, outliers and pipeline construction. | Preprocessing report + validated mlr3pipelines graph. |
| Week 3 - Modeling | Baseline, linear, regularized, tree and ensemble learners; common resampling and measures. | Benchmark results + model analysis. |
| Week 4 - Evaluation | Cross-validation diagnostics, error analysis, residual analysis, feature importance and robustness checks. | Evaluation report + comparison visuals. |
| Week 5 - Tuning + XAI | Tune finalists, use nested resampling where practical, implement SHAP/LIME and compare explanation behavior. | Tuning archive + XAI analysis. |
| Week 6 - Shiny + Finalization | Build Shiny UI, prediction flow, explanation display, documentation, final report and GitHub polish. | Working Shiny app + README + final report + repository. |

**Execution discipline**

Do not build the Shiny application before the final predictive pipeline is stable. The project should proceed from data understanding to reproducible modeling, then explainability, then UI integration.

# 20. Repository and Deliverables

## 20.1 Recommended Repository Structure

```
Explainable-House-Price-Prediction-R-mlr3/
|-- R/
|   |-- 01_setup.R
|   |-- 02_data_audit.R
|   |-- 03_eda.R
|   |-- 04_preprocessing.R
|   |-- 05_feature_engineering.R
|   |-- 06_benchmark_models.R
|   |-- 07_tuning.R
|   |-- 08_evaluation.R
|   |-- 09_explainability.R
|   `-- 10_utils.R
|-- app/
|   `-- app.R
|-- reports/
|-- figures/
|-- data/                # only if license/source terms permit local copy
|-- README.md
|-- renv.lock
|-- .gitignore
`-- LICENSE
```

## 20.2 Deliverables

- Complete R implementation with reproducible setup.
- Data-quality and preprocessing analysis.
- mlr3 benchmark and tuning experiments.
- Final model and documented selection rationale.
- SHAP + LIME explanation artifacts.
- Working Shiny application.
- Final technical report and research discussion.
- Professional README with installation, data source, workflow, results, limitations and future scope.
- Version-locked R environment using `renv`.

# 21. Acceptance Criteria

**1.** The canonical Ames Housing task loads successfully from `mlr3data` and the project records the environment/version context.

2. The raw data audit identifies the documented constant, duplicate and duplicate-scale examples before those features are removed.
3. The preprocessing workflow addresses missingness and categorical variables without training on validation/test information.
4. At least four diverse regression learners are benchmarked on common resampling splits.
5. At least one advanced model is tuned and evaluated with a reproducible search configuration.
6. The final report contains MAE, RMSE and R-squared results plus relevant diagnostic plots.
7. SHAP and LIME explanations can be generated for selected predictions and are presented as model-attribution explanations rather than causal statements.
8. The Shiny application accepts house information, returns a prediction and displays the main contributing factors.
9. All final claims are supported by experiment outputs or cited sources; no fabricated results or unsupported data-quality claims are included.
10. The project repository is understandable to an external reviewer who has not seen the development process.

## 22. References

[1] Bischl, B., Lang, M., Kotthoff, L., Schiffner, J., Richter, J., Studerus, E., Casalicchio, G., & Jones, Z. M. (2016). mlr: Machine Learning in R. Journal of Machine Learning Research, 17(170), 1– 5. Source

[2] Lang, M., Binder, M., Richter, J., Schratz, P., Pfisterer, F., Coors, S., Au, Q., Casalicchio, G., Kotthoff, L., & Bischl, B. (2019). mlr3: A modern object-oriented machine learning framework in R. Journal of Open Source Software, 4(44), 1903. DOI: 10.21105/joss.01903. Source

[3] Binder, M., Pfisterer, F., Lang, M., Schneider, L., Kotthoff, L., & Bischl, B. (2021). mlr3pipelines - Flexible Machine Learning Pipelines in R. Journal of Machine Learning Research, 22(184), 1– 7. Source

[4] Bischl, B., Sonabend, R., Kotthoff, L., & Lang, M. (Eds.). (2024). Applied Machine Learning Using mlr3 in R. CRC Press. Source

[5] mlr3 Book, Chapter 9: Preprocessing. The chapter documents the adapted Ames Housing dataset, its intentional preprocessing problems and the associated mlr3pipelines methods. Source

[6] mlr3data reference: House Sales in Ames, Iowa. 2,930 observations; 80 predictors; target `Sale_Price`. Source

[7] De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. Journal of Statistics Education, 19(3). Source

[8] mlr3tuning documentation. Used for hyperparameter optimization and nested resampling; current ecosystem documentation describes random search and extension packages for methods such as Bayesian optimization and Hyperband. Source

[9] Posit Shiny documentation. Used for the interactive R web application architecture. Source

### Reference Notes and Interpretation Boundaries

- The mlr3 paper describes the framework architecture and ecosystem rather than a single domain application; the project therefore uses it as an implementation foundation, not as a dataset-performance benchmark.
- The mlr3 paper does not contain a formal limitations/future-work section. The implementation limitations in this PRD are therefore explicitly separated into historical framework lessons, practical project constraints, and dataset/model limitations.

- The adapted Ames dataset is intentionally modified for teaching; project reports must preserve that distinction from the original Ames data source.

**Project decision**

Final dataset: `mlr3data::ames_housing`. Final task: house-sale-price regression. Core framework: `mlr3` + `mlr3pipelines`. Primary learning objectives: R data science, preprocessing and ML. Secondary objectives: SHAP/LIME explainability and Shiny deployment.